

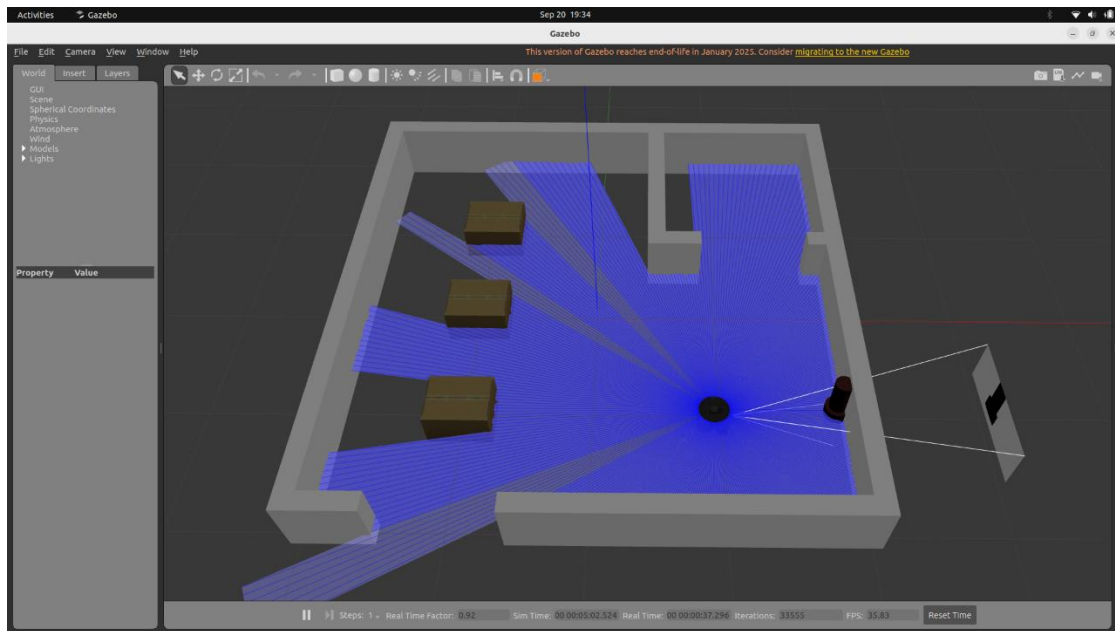
ROS Task Documentation

- ROS2 + Nav2 stack implementation

- Naval Selvan. S

Work on Simulation:

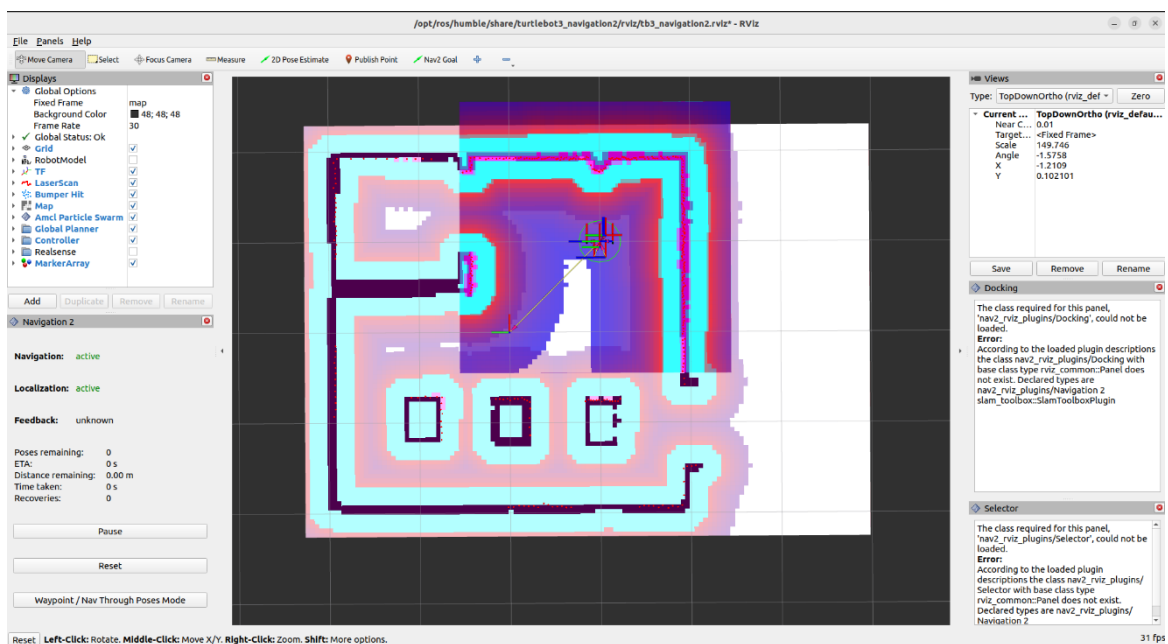
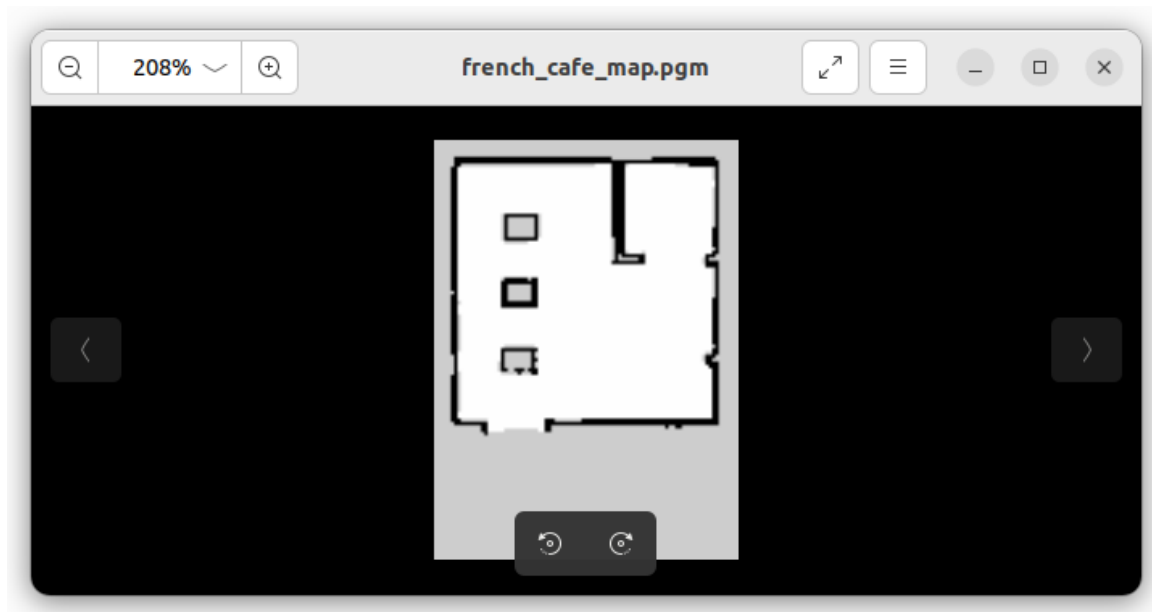
- A custom world was created in Gazebo with the simple tools available from its library. The arena created is of 4x4 tile sq. dimension with kitchen area at its upper right corner. The map's layout sits on a floor plane with the plane's origin as its centre.
- The black standing pipe in the right wall is the docking station for bot and it sits at [1,-1] on the floor plane as its home point.
- Turtlebot3 has been chosen for this simulation as its one of the bot models which is a widely used AMR model in labs and work places for basic control and code implementation purposes. Also, as the problem statement mainly focuses on the navigational approach, we are going with known bot and a simple world layout.
- Turtlebot3's "Waffle" model is used in this simulation.
- Three wooden boxes are used to represent cafe tables despite availability of tables from the Gazebo library. This is because the table's leg marking in the mapped environment doesn't show the actual area covered by the table. So, replacing them with boxes made better sense in terms of visualisation.
- Table1, Table2 and Table3 sit at [-1,1], [-1,0], [-1,-1] floor plane coordinates respectively, all placed equidistant to each other.



Mapping the environment:

- Firstly, a ROS2 workspace named “cafe_robot_ws” was created. In the source folder, all the simulation related files required for Turtlebot3 (available in GitHub) were added. Later it was colcon built for the package compilation.
- We have named the world model as “cafeworld.world” and saved it in the ‘worlds’ folder inside the “turtlebot3_gazebo” package.
- Later our world was mapped in RViz using a cartographer launch file that came along with the bot's simulation files. This gives us a perfectly mapped world layout which later is very useful in programming bot's navigation inside the actual arena.
- After mapping, the maps's .pgm and .yaml files are stored inside turtlebot3_gazebo package in maps folder. Its named as “french_cafe_layout.yaml” .
- In the mapped layout, the bot's initial starting position is considered as the origin for reference. So here on, we'll talk about the positions in terms of mapped layout frame.

- So $[1, -1]$ in the floor plane's coordinate system becomes our new reference origin.
- Turtlebot3 is parked with too much space in front of its docking point (black standing pipe) to avoid spawning related issues when the launch file is run.



Code Explanation

Python file name: *final_delivery.py*

File location: *src/turtlebot3_gazebo/script/final_delivery.py*

- In Problem statement, bot movement between all given points (home, kitchen, table1, table2, table3) entirely depends on commands (food orders & confirmations) we manually enter during the simulation. We give this in as messages with message type “OrderEvent” to get published onto a topic named “order_events”. This has a subscriber that sits inside our python navigation file.
- “OrderEvent” is a custom message type with two string elements - event_type , table_id which is present in the “own_interfaces” folder inside the source folder.
- Let us go through the code and its functioning.
- Code functions by receiving our commands with help of its subscriber node and sharing it with the function blocks that work with various ‘if else’ cases for calling in the right navigation 2 stack methods at the right time.
- First, we set the initial pose of the bot to the origin of the map frame with help of SetInitialPose(). We use a dictionary named COORDS to get the coordinates of all our desired spots when needed.

- SetInitialPose() takes in positional and orientational values stored together in a message type named PoseStamped from geometry_msgs interface. The orientational values are sent as quaternion angles. So, we give in our values to create_pose_stamped() func which does the job of angle conversion to Euler and assign positions
- Then we go with creating a 'OrderManager' class whose functions do most of the code's job. There are five important functions to look upon.
- Inside we initiate 2 important attributes of this class - inbox[] and orders{ }. Former stores the incoming message values that are published and the latter stores the table IDs with their current state (confirm/cancel/delivered/not ordered yet).
- Entering the class, we do a 2D pose estimate of bot in mapped layout in RViz using SetInitialPose(). Then we initiate a subscriber subscribed to "order_events" topic and its callback func does the job of adding the incoming message value into inbox[].

1) move_robot()

- This takes the job of navigating bot to the given destination with goToPose(pose)
- Upon reaching the destination, it stops navigation and print the result (success/failed) with getResult() function.

2) wait_for_event()

- This func waits for confirmation or cancellation reply when bot reaches the destination. Once it gets the reply, if it's 'confirm' then it makes it proceed further, else it makes bot go to the kitchen and then to home.

- It returns the choice of the user that's given in or 'timeout' if user fails to respond on time.

3) wait_for_order_trigger()

- After initialising the bots position at start, it waits for the user command to start performing the task of delivering food. So, we give the order through the terminal by publishing our order details to the topic.
- It has a time out limit of 60 seconds. so, failing to send proper commands will reach the end of the program.
- Command needs to have word 'order' as choice in event_type and proper table ID. Else the command won't be considered.
- This returns a bool value at the end. True for receiving and false for no response.

4) check_table_status()

- This checks whether the given table ID has received any message for its order cancellation before preparing to move to its respectable table position.
- It checks inbox[] for the word "cancel" in the list of choices, if yes then it removes its entry from inbox[] and returns "cancelled". Else it returns None.

5) handle_delivery_round()

- This function does its job by calling all of the above mentioned functions along with proper if else code blocks to check on various problem conditions given.

- This gets called in the main() block by giving in all of the table IDs. First, it calls wait_for_order_trigger() and gets in all the orders.
- If no orders are received then it stays at home position and the program comes to an end.
- Then, bot starts to move towards kitchen. After reaching, it waits for confirmation from the user by calling wait_for_event(). Positive confirmation makes it proceed further towards the table. Else it returns back to home position.
- We update orders{ } with all table_id and their current updated choices.
- Now, it enters into a for loop with table IDs as iterators. Here it begins by calling check_table_status(). After getting return value from it , it does a proper updation of orders{ } mentioning whether it's still in the “confirm” / “cancel” / “No Order passed” state.
- After this , it calls move_robot() function to move bot to the tables with “confirm” choices by checking it with orders{ }.
- wait_for_event() is called each time when bot reaches the table, then confirmation is seeked, irrespective of confirmation value, it proceeds to next table if there is any. At the end if it had received cancellation from any table upon reaching previously, it would reach back to the kitchen again and then finally reaches home. Else it would directly go for home position.
- In the main() block, you initiate a node for OrderManager class and then we just go for the handle_delivery_round() function call.

Simulation

1. We start the gazebo and navigation launch files in terminals.

- Then, we get into our workspace , source it, and we launch turtlebot3_my_world.[launch.py](#)

- For that, we type in the following terminal command:

```
ros2 launch turtlebot3_gazebo turtlebot3_my_world.launch.py
```

2. Now load the mapped environment in Rviz by launching "[navigation2.launch.py](#)" by providing the map's address with respect to the terminal to load.

```
"ros2 launch turtlebot3_navigation2 navigation2.launch.py use_sim_time:=True  
map:=src/turtlebot3_gazebo/maps/french_cafe_map.yaml"
```

3. We have a gazebo and rviz parallely running at the back. Go to another terminal now. Get into the script folder present inside turtlebot3_gazebo package. Now run the final_delivery.py file. Since it's executable you can run it from the terminal.

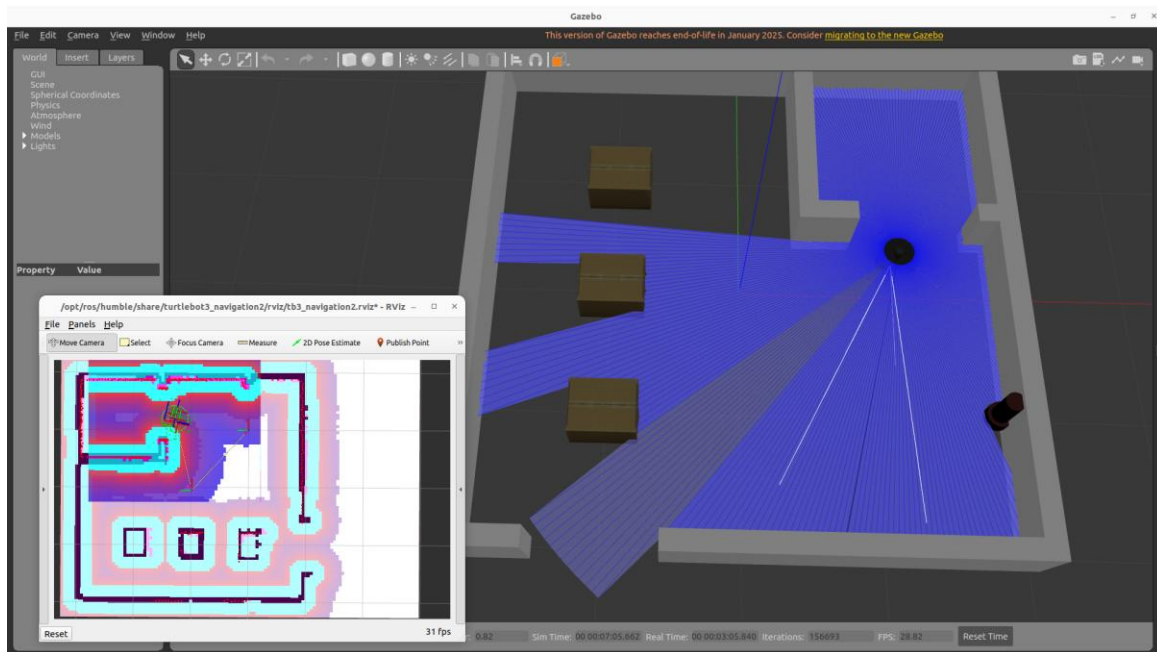
```
./final_delivery.py
```

4. Now in the new terminal, get into the workspace and source it. Check for the "OrderEvent" message type used in code, to know about its attributes.

```
ros2 interface show own_interfaces/msg/OrderEvent
```

5. Now, you can initiate the navigation by giving orders for the table in the form of messages publishing on to a topic.

```
ros2 topic pub - -once /order_events own_interfaces/msg/OrderEvent "{event_type:  
'order', table_id: 'table1'}"
```



Remember

- You are supposed to give only “order” as choice in event_type when you start by typing in table orders.
- You can add orders for others consecutively until the bot's confirmation time limit gets over after reaching “Kitchen” state. i.e., you can give in order even during the bot's wait time in the kitchen.
- For confirming/ cancelling, you are supposed to change the event_type to either “confirm” or “cancel” and table_id to its respective table number.